REACT · DERS MATERYALI

Context API ve useContext

Prop drilling sorunu, global state ve sepet uygulaması

```
// main.jsx
<BasketProvider>
  <App />
</BasketProvider>

// herhangi bir bileşende
const { basket } =
  useContext(BasketContext);
```

PLAN

Bugün neler öğreneceğiz?

01

Prop drilling

Veriyi kademe kademe aktarmanın sorunu

02

Context API

createContext, Provider, useContext

03

BasketContext

Sepet state'i ve üç fonksiyon

04

Bileşenlerde kullanım

Card, Header, Basket, BasketInfo

05

İyi pratikler

Render davranışı, ayrı context'ler

06

Hatalar ve alıştırma

Sık hatalar, özet, ödev

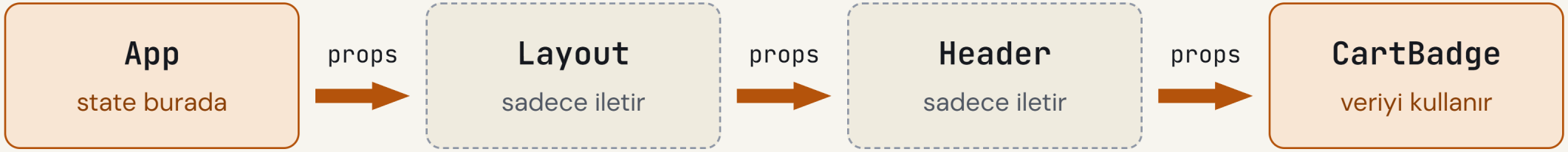
01

Prop drilling sorunu

Derindeki bir bileşen bir veriye ihtiyaç duyunca ne olur?

01 · PROP DRILLING

Prop drilling nedir?



Sepet state'i **App** içinde tutulur, ama sepet sayısını gösteren **CartBadge** ağacın en altındadır. Veri, kullanmayan iki bileşenden geçirilmek zorunda kalır.

Prop drilling: veriyi kullanmayan ara bileşenlerden geçirerek derindeki bileşene ulaştırmak.

01 · PROP DRILLING

Prop drilling neden sorun?

```
// App.jsx
<Layout basket={basket} />

// Layout.jsx
<Header basket={basket} />

// Header.jsx
<CartBadge basket={basket} />
```

Ara bileşenler kullanmadıkları bir **prop'u** almak zorunda kalır.

Verinin adı veya yapısı değişince **tüm zincir** güncellenir.

Bileşen sayısı arttıkça kod **okunmaz** hale gelir.

Örnek projede sepet verisine Header, Card, Basket, BasketCard ve BasketInfo ihtiyaç duyuyor.

01 · PROP DRILLING

Çözüm seçenekleri

Yöntem	Ne zaman?	Not
Props	Veri bir iki kat aşağı iniyorsa	En basit yöntem
Context API	Çok sayıda bileşen aynı veriyi kullanıyorsa	React'in içinde, ek kurulum yok
Redux, Zustand gibi kütüphaneler	Büyük ve karmaşık state'lerde	Ek paket gerekir

Bu derste Context API kullanıyoruz: ürün listesi ve sepet.

02

Context API

Veriyi bileşen ağacına tek bir yerden yayınlamak

02 · CONTEXT API

Context nedir?

Context, veriyi props olmadan **doğrudan ihtiyaç duyan bileşene** ulaştırır.

Veri **Provider**'da tutulur, bileşenler **useContext** ile abone olur.

Aradaki bileşenlerin **hiçbir prop taşıması** gerekmez.



02 · CONTEXT API

Context’in dört adımı

1

Oluştur

`createContext()`

Boş bir context nesnesi üretir.

2

Sağla

`Provider value`

Veriyi ve fonksiyonları yayınlar.

3

Sarmala

`main.jsx`

Uygulamayı Provider ile çevreler.

4

Kullan

`useContext()`

Bileşen veriye abone olur.

Projedeki BasketContext de bu dört adımla kuruldu.

1. Adım: createContext

```
// context/basket-context.jsx
import { createContext } from "react";

export const BasketContext =
  createContext();
```

createContext bir context nesnesi döndürür. Provider ve useContext bu nesneyi kullanır.

Nesne **export** edilir; diğer dosyalar **useContext(BasketContext)** yazabilsin diye.

Parantez içine varsayılan değer yazılabilir. Boş bırakılırsa **undefined** olur.

2. Adım: Provider bileşeni

```
export const BasketProvider =
  ({ children }) ⇒ {
  const [basket, setBasket] =
    useState([]);

  // sepet fonksiyonları da burada

  return (
    <BasketContext.Provider
      value={{ basket }}>
      {children}
    </BasketContext.Provider>
  );
};
```

Provider da sıradan bir **bileşendir**; state içinde yazılır.

value: tüm tüketicilere aktarılacak veri. Genelde bir nesnedir.

children: Provider'ın sarmaladığı bileşenler.

children prop'u nasıl çalışır?

MAIN.JSX

```
<BasketProvider>
  <App />
</BasketProvider>
```

children



BASKET-CONTEXT.JSX

```
<BasketContext.Provider
  value={{ basket }}>
  {children}
</BasketContext.Provider>
```

Etiketin içine yazılan her şey, bileşene **children** adıyla prop olarak gelir. Provider, **App**'i bu yolla alıp kendi çıktısının içine koyar.

App'in içindeki tüm bileşenler Provider'ın içinde olduğu için veriye erişebilir.

3. Adım: Uygulamayı sarmalama

```
// main.jsx
import { BasketProvider } from
  "./context/basket-context.jsx";

createRoot(
  document.getElementById("root")
).render(
  <BasketProvider>
    <App />
  </BasketProvider>
);
```

Yalnızca Provider'ın **içindeki** bileşenler context'e erişebilir.

Sarmalama genelde **main.jsx**'te yapılır; böylece tüm uygulama kapsanır.

Birden fazla context olursa Provider'lar **iç içe** yazılır.

4. Adım: useContext ile erişim

```jsx
// components/Header.jsx
import { useContext } from "react";
import { BasketContext } from
  "../context/basket-context";

const Header = () => {
  const { basket } =
    useContext(BasketContext);
  // Sepet ({basket.length})
};
```

useContext, en yakın Provider'ın **value** değerini döndürür.

Destructuring ile yalnızca **ihtiyacın olan** alanları al.

value değişirse bileşen **yeniden render** edilir.

03

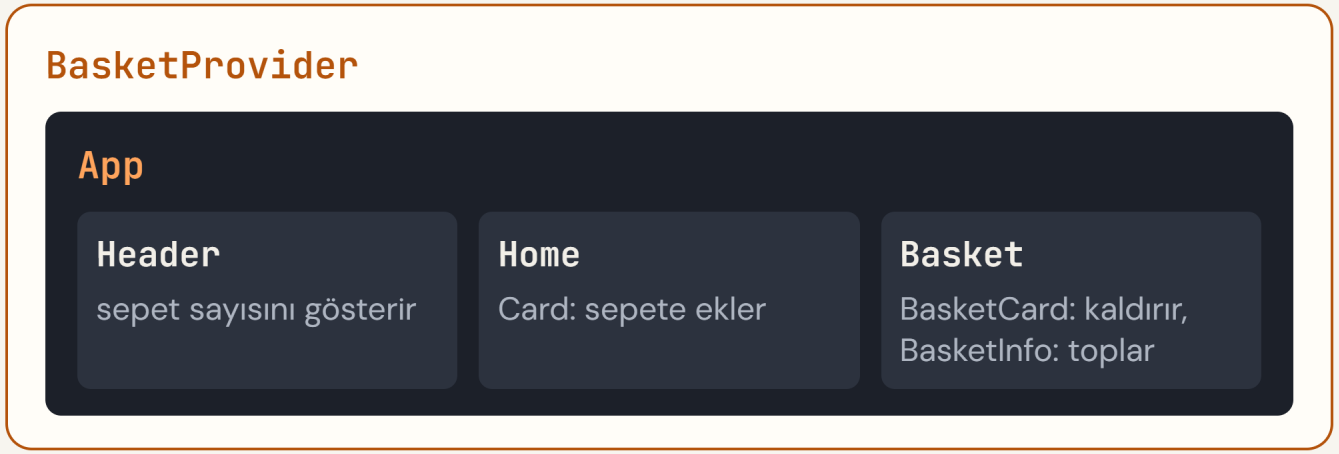
BasketContext

Sepeti ve sepeti değiştiren fonksiyonları tek yerde yazmak

O3 · BASKETCONTEXT

Örnek projenin yapısı

```
src/
  context/
    basket-context.jsx
  data/
    products.js
  components/
    Header, Card,
    BasketCard, BasketInfo
  pages/
    Home.jsx, Basket.jsx
  App.jsx, main.jsx
```



Provider: state ve fonksiyonlar

```
export const BasketProvider =
  ({ children }) ⇒ {
  const [basket, setBasket] =
    useState([]);

  const addToBasket = (product) ⇒ {…};
  const removeFromBasket = (id) ⇒ {…};
  const clearBasket = () ⇒ {…};

  return (
    <BasketContext.Provider value={{
      basket, addToBasket,
      removeFromBasket, clearBasket
      }}>
```

Alan	Tür	Kullananlar
basket	dizi	Header, Card, Basket
addToBasket	fonksiyon	Card
removeFromBasket	fonksiyon	BasketCard
clearBasket	fonksiyon	Basket

Context ile yalnızca veri değil, veriyi değiştiren fonksiyonlar da paylaşılır.

03 · BASKETCONTEXT

Ekle, kaldır, temizle

```
const addToBasket = (product) => {
  const found = basket.find(
    (item) => item.id === product.id
  );
  if (!found) setBasket([...basket, product]);
};

const removeFromBasket = (id) => {
  setBasket(basket.filter((item) => item.id !== id));
};

const clearBasket = () => setBasket([]);
```

addToBasket: ürün sepette yoksa yeni diziye ekler.

removeFromBasket: filter ile ürünü sepetten direkt kaldırır.

clearBasket: boş dizi vererek sepeti sıfırlar.

State'i doğrudan değiştirme (push, splice). Her seferinde yeni dizi üret: spread veya filter.

04

Bileşenlerde kullanım

Card, Header, Basket ve BasketInfo sepete nasıl ulaşır?

04 · BILEŞENLERDE KULLANIM

Card: sepete ekle butonu

```
const { basket, addToBasket } =
  useContext(BasketContext);

// bu ürün sepette var mı?
const inBasket = basket.some(
  (item) ⇒ item.id ≡≡ product.id
);

<button
  disabled={inBasket}
  onClick={() ⇒ addToBasket(product)}
>
  {inBasket ? "Sepette" : "Sepete Ekle"}
</button>
```

Context'ten hem **veri** hem **fonksiyon** alındı.

Ürün sepetteyse buton **Sepette** yazar ve kapanır.

Tıklayınca sepet güncellenir; **Header** sayacı da değişir.

Header: sepet sayacı

```
const Header = () => {
  const { basket } =
    useContext(BasketContext);

  return (
    <NavLink to="/sepet">
      Sepet ({basket.length})
    </NavLink>
  );
};
```

Header, Card'dan **çok uzakta** ama aynı sepete bakar.

Hiç **prop** almaz; sepet sayısı doğrudan context'ten gelir.

Ürün eklenince ya da kaldırılınca sayaç **otomatik** güncellenir.

BasketCard: ürünü sepetten kaldır

```
const { removeFromBasket } =
  useContext(BasketContext);

<h4>{product.title}</h4>
<h3>{product.price}₺</h3>

<button onClick={() ⇒
  removeFromBasket(product.id)}
>
  Kaldır
</button>
```

Kaldır butonu ürünü sepetten **direkt** çıkarır.

Fonksiyon **BasketContext**'te yazılı; BasketCard sadece çağırır.

Ürün kalkınca Card'daki buton **yeniden** "Sepete Ekle" olur.

BasketInfo: sepetten hesaplanan değerler

```
const { basket } =
  useContext(BasketContext);

// toplam fiyat
const totalPrice = basket.reduce(
  (total, item) ⇒ total + item.price,
  0
);

// ekranda
Ürün Adedi: {basket.length}
Toplam Fiyat: {totalPrice}₺
```

Toplam fiyat ayrı bir state değil; **basket**'ten her render'da hesaplanır.

Doğru kaynak **tek**: basket. Çifte state tutmak tutarsızlık doğurur.

reduce, diziyi tek bir değere indirger.

05

İyi pratikler

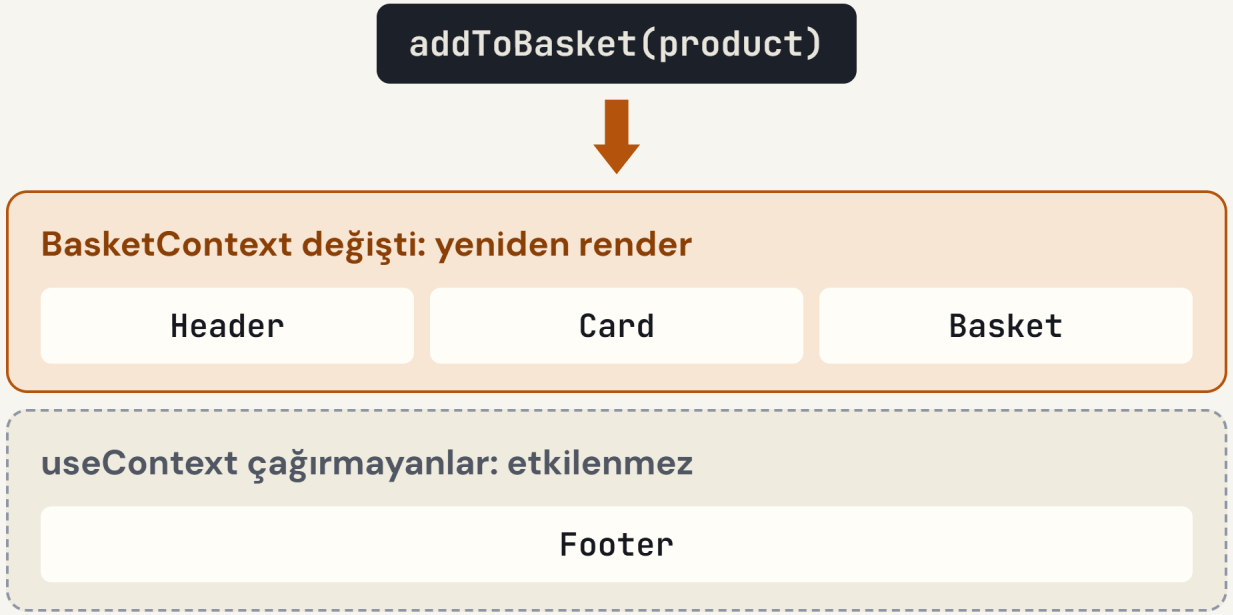
Context ne zaman yeniden render eder, ne zaman kullanılmalı?

Context değişince ne olur?

value değişince o context'i **useContext** ile okuyan tüm bileşenler yeniden render edilir.

Sepete ürün eklenince **BasketContext**'i okuyanlar güncellenir.

Context'i **okumayan** bileşenler etkilenmez.



05 · İYI PRATIKLER

Yeni konu gelirse: ayrı context

	Tek context	Ayrı context'ler
Örnek	Sepet ve tema tek value'da	BasketContext ve ThemeContext
Sepet değişince	Temayı okuyanlar da render olur	Yalnızca sepeti okuyanlar render olur
Dosyalar	Büyür ve karışır	Küçük, tek amaçlı
Bu projede	Sepet tek konu	Yeni konu gelince yeni context

Kural: birlikte değişen veriyi aynı context'te, ilgisiz veriyi ayrı context'te tut.

05 · İYI PRATIKLER

Context ne zaman kullanılır?

Kullan

- Tema, dil, oturum açmış kullanıcı
- Sepet, favoriler
- Birçok sayfanın kullandığı API verisi

Gerek yok

- Bir iki bileşenin kullandığı veri: props
- Form input gibi yerel state: useState
- Çok sık değişen büyük state

Önce props ve useState ile başla. Veri gerçekten uzağa taşınıyorsa context ekle.

React 19 ile Context

BU DERSTE

```
<BasketContext.Provider
  value={value}>
  {children}
</BasketContext.Provider>

const { basket } =
  useContext(BasketContext);
```

REACT 19

```
<BasketContext
  value={value}>
  {children}
</BasketContext>

const { basket } =
  use(BasketContext);
```

React 19'da context doğrudan Provider gibi kullanılabilir, **use** ise useContext yerine geçer ve **koşul içinde** de çağrılabilir.

Derste Provider ve useContext ile devam ediyoruz. İkisi de çalışmaya devam eder.

DIKKAT

Sık yapılan hatalar

Provider'ı unutmak

Bileşen Provider'ın dışındaysa useContext undefined döner, destructuring hata verir.

Hook'u import etmemek

useState veya useContext import edilmezse "is not defined" hatası alınır.

State'i doğrudan değiştirmek

basket.push() yapma. Yeni dizi üret, setBasket ile ver.

Her şeyi tek context'e koymak

İlgisiz veriler birbirinin render'ını tetikler. Konuya göre ayır.

value'da olmayan alan

Provider'ın value'sunda yoksa useContext ile okunan alan undefined gelir.

Hata ayıklarken

React DevTools'ta Components sekmesinde context değerlerine bakabilirsin.

ÖZET

Bugün öğrendiklerimiz

Sorun

Prop drilling: veri, kullanmayan bileşenlerden geçirilir.

Dört adım

createContext, Provider, uygulamayı sarmala, useContext.

Uygulama

BasketContext sepeti tutar; ekle, kaldır ve temizle fonksiyonlarını paylaşır.

Dikkat

Konuya göre ayrı context, yeni nesne üret, Provider dışında kullanma.

ALIŞTIRMA

Sıra sizde

1

Favoriler context'i

FavoriteContext yaz: ürünü favoriye ekle ve çıkar. Header'da favori sayısını göster.

2

Sepeti kalıcı yap

basket değişince localStorage'a yaz, sayfa açılırken oradan başlat.

3

Özel hook yaz

useBasket() hook'u BasketContext'i döndürsün. Tüm bileşenlerde onu kullan.

Bonus: Sepete miktar ekle. Aynı ürün tekrar eklenince adedi artsın, - ve + butonları olsun.